L L M E N G I N E E R I N G

Week 01

Getting Started

Thursday, 15 October 2026

Interactive version: https://llm.strueber.dev/#/week-01

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

The Gap

1

> **I believe that every university that has an AI group should train their own large language model.**

Raia Hadsell
VP of Research, Google DeepMind
Frontier AI Innovation Summit, Stuttgart, 2026 (Hadsell, 2026)

▼ Why train your own model

"I believe that every university that has an AI group should train their own large language model." Raia Hadsell, VP of Research at Google DeepMind, said this on a panel at the Frontier AI Innovation Summit in Stuttgart in October 2026 (Hadsell, 2026). She added that such a model will not come near the frontier, but that training one is the only way to learn how to build a language model and how it works.

That is the idea behind this course. Most courses teach the theory and the formulas. You learn to read a Transformer diagram, derive backpropagation and write down the equations of self-attention. Few courses teach how to implement these pieces, how to make them efficient, and which trade-offs each design choice brings. Production needs those skills. It asks what you do when a training run diverges after a configuration change, when an evaluation regresses, or when inference is too slow. Knowing about a system and being able to diagnose one are different skills. This course trains the second.

Academic compute is scarce

- An envied lab server: 4 GPUs, €10,000
- Typical academic: 4 GPUs, for days (Khandelwal et al., 2025)
- Llama 3 405B: up to 16K H100s (Grattafiori et al., 2024)
- Using compute well sets you apart

▼ How much compute academics have, and how much industry has

At my former university, a junior professor got funding to build a small lab server with 4 Titan RTX GPUs. It cost about €10,000, and colleagues envied the professor for it.

A survey shows how typical such a machine is. In April 2024, Khandelwal and colleagues collected answers from 50 researchers at 35 institutions about the compute available to them per person or per project. The typical respondent could use 4 GPUs for days at a time. At the high end, some could use 4 GPUs for weeks, or 8 to 16 GPUs for days. Only 10% reported access to H100 GPUs, the newest data-center generation at the time (Khandelwal et al., 2025). The authors reached their respondents over three weeks by word of mouth, Twitter and inter-university Slack channels. The results therefore describe the researchers who answered, and they may not represent academia as a whole.

Frontier labs work at a different scale. Llama 3 405B was trained on up to 16K H100 GPUs (Grattafiori et al., 2024).

We draw one conclusion from these numbers. When every run is expensive, you need reliable answers from few runs, and using your compute well sets you apart. The Challenge tab shows how to do that.

3

Frontier-lab interviews test engineering

- OpenAI, Anthropic, Google DeepMind, Meta Superintelligence Labs
- Mostly parallelization and hardware utilization
- Weeks 2–6 teach both

▼ What frontier labs ask about

Friends of mine recently interviewed for engineering roles at OpenAI, Anthropic, Google DeepMind and Meta Superintelligence Labs. Most of the questions they got were about two topics. Parallelization splits the work of training a model across many GPUs. Hardware utilization is the share of each GPU's peak throughput that a workload uses.

Weeks 2 to 6 of this course teach both. Week 2 counts the floating-point operations (FLOPs) and the memory a model needs, the numbers both topics start from. Week 4 measures hardware utilization. Week 5 covers mixed precision, which runs most of the arithmetic in a lower-precision number format and keeps the values that need it in higher precision. Week 6 covers parallelization.

4

The goal:

**the skills to start a PhD in a top program,
join a frontier lab such as Google DeepMind,
or found a deep-tech startup,
and be productive from day 1.**

▼ The goal of this course

The goal: the skills to start a PhD in a top program, join a frontier lab such as Google DeepMind, or found a deep-tech startup, and be productive from day 1.

In practice, you should leave able to orient yourself in a real LLM training codebase, recognize the patterns that recur across such codebases, and judge why a system made the trade-offs it did.

What does it take to be productive from day 1?

- Where the field stands
- Derisk every decision before scaling
- How this course trains you
- The one objective: next-token prediction

▼ Four parts of one answer

What does it take to be productive from day 1? The answer has four parts. First, you know where the field stands: how language models have developed since 2017, and which weeks of this course build its pieces (the History tab). Second, you derisk every decision before you scale it, by testing it first in a small experiment (The Challenge tab). Third, you practice in a loop of specification, implementation, measurement and feedback, which is how this course trains you (the rest of this tab and the Course Overview tab). Fourth, you understand the one objective that everything else builds on: predicting the next token (the Next-Token Prediction tab).

Every week runs one loop



- One config-driven codebase grows all semester
- Coding starts in Week 2

▼ Practice with feedback, every week

Ericsson describes **deliberate practice** as focused work on a specific weakness, guided by feedback and repeated with adjustment (Ferlazzo et al., 2016). Each week of this course runs a loop built on that idea. The loop is our own design: the research motivates it, but it studied other skills and never tested this loop.

The loop has four stations. A specification states what your code must do, in terms a test can check, before you change any code. For each exercise, the course gives you the specification: written instructions and a test suite. Your implementation then produces behavior you can run and inspect. An argument can sound plausible and still be wrong. Tests and measurements show where that behavior differs from what you intended. In the tutorial you get feedback on that evidence, and you revise the implementation. The next exercise's specification builds on it. The loop turns theory into experimental engineering: in each cycle you make a claim, produce evidence, learn from the difference and revise, and the evidence decides what you do next.

All exercises extend one codebase, which grows through the whole semester. It is config-driven: every hyperparameter of a run comes from a configuration file. Week 1 sets up the concepts and the environment, and coding starts in Week 2.

History

Seven eras since 2017

7

- Core architecture: 2017 (Vaswani et al., 2017)
- Many milestones map to a course week

▼ How to read the timeline, era by era

The history of large language models is compressed into a few years. Their core architecture, the Transformer, was introduced in 2017 in "Attention Is All You Need" (Vaswani et al., 2017). Since then the field has moved through seven eras, each building on the last. The timeline after this slide is selective. It follows one path from the Transformer to today's agents, and where a week of this course builds a piece, its card names that week. Each card picks a thread the course returns to. The timeline leaves most models out and compares none of them under controlled conditions. Two card claims have no source here: the wave of open tooling after LLaMA, and the coding-agent products of 2025. Check them before you rely on them.

On a wide screen, a plot of METR's measurements stays beside the cards. On a narrow screen it is hidden. It shows each model's release date against its task horizon. The vertical axis is logarithmic: each gridline is ten times the one below, and the right-hand labels mark a second, a minute, an hour and a day. On such an axis, a straight line means a fixed doubling time. A point's color shows its developer: OpenAI, Anthropic or Google. The points carry no labels, but hovering over one shows its model and horizon. Only a few of them, such as GPT-2, GPT-3, o1 and o3, also have a card. The dashed vertical line follows the card you are reading, and a model's point appears once the line reaches its release date, so the plot fills in as the history moves forward.

METR measures a model's **time horizon**: the length of task, measured by the time a skilled human takes to do it, that the model completes with a 50% success rate (Kwa et al., 2025). Its original paper estimated that the horizon of frontier models has doubled about every seven months since 2019 (Kwa et al., 2025). The updated results, Time Horizon 1.1, give 188 days, about six months, over all models measured (METR, 2026). The dashed diagonal line takes that doubling time as its slope. We fitted its height to the points at or below 16 hours with the slope held fixed, so the slope is METR's and the placement is ours. From GPT-2's roughly 3 seconds in 2019 to about 17 hours in 2026, the points rise by more than four orders of magnitude and stay close to the dashed diagonal line, so the horizon grows exponentially (METR, 2026). METR calls measurements above 16 hours unreliable with its current task suite, and its doubling times exclude them (METR, 2026). The plot describes METR's task suite only, whose tasks and evaluation conditions the plot does not show, so do not generalize it beyond that setup. Its points come from three developers, so it says nothing about the open-weight models on the cards. Nor is it a ranking of the models.

Recurrent networks such as LSTMs read a sequence one token at a time, and each step depends on the one before. The positions of a sequence therefore cannot be trained in parallel, and information from a distant token has to pass through every step in between, which makes long-range dependencies hard to capture (Vaswani et al., 2017). The Transformer replaced recurrence with self-attention, so a whole sequence trains at once (Vaswani et al., 2017). In its decoder, which you build in Week 2, each token attends to itself and to every earlier token (Vaswani et al., 2017). GPT (2018) pretrained such a decoder on unlabeled text, then fine-tuned it per task, improving on the state of the art in 9 of the 12 tasks studied (Radford et al., 2018). GPT-2 (2019) kept largely the same architecture, scaled up model and data, and translated, summarized and answered questions zero-shot, without task-specific training (Radford et al., 2019).

Scaling came next (2020 to 2022). GPT-3, with 175B parameters, learns a task from a few examples in the prompt (Brown et al., 2020). Week 7 returns to the power laws that Kaplan et al. found for the loss in model size, dataset size and compute, over the ranges they studied (Kaplan et al., 2020). Wei et al. reported abilities that appear in larger models but not in smaller ones, and called them emergent (Wei et al., 2022). Schaeffer et al. showed that the metric can produce much of that picture, because a nonlinear or discontinuous metric turns smooth improvement into a sudden jump (Schaeffer et al., 2023). In 2022, the Chinchilla study found that the large models of the time were undertrained: they needed more training data per parameter (Hoffmann et al., 2022). Compute-optimal training grows parameters and training tokens roughly equally with the compute budget, which changed the recommended split between model size and data (Hoffmann et al., 2022).

ChatGPT, released in November 2022, made post-training visible. OpenAI fine-tuned it from a GPT-3.5 model with supervised fine-tuning and reinforcement learning from human feedback (OpenAI, 2022). The recipe was InstructGPT's: fine-tuning on human demonstrations, then reinforcement learning against a reward model learned from human rankings of outputs (Ouyang et al., 2022). Compared with GPT-3, what changed was post-training, which turns a base model into an assistant and which Week 12 covers. By a UBS estimate from Similarweb data, ChatGPT had 100 million monthly active users in January 2023, two months after launch (Hu, 2023).

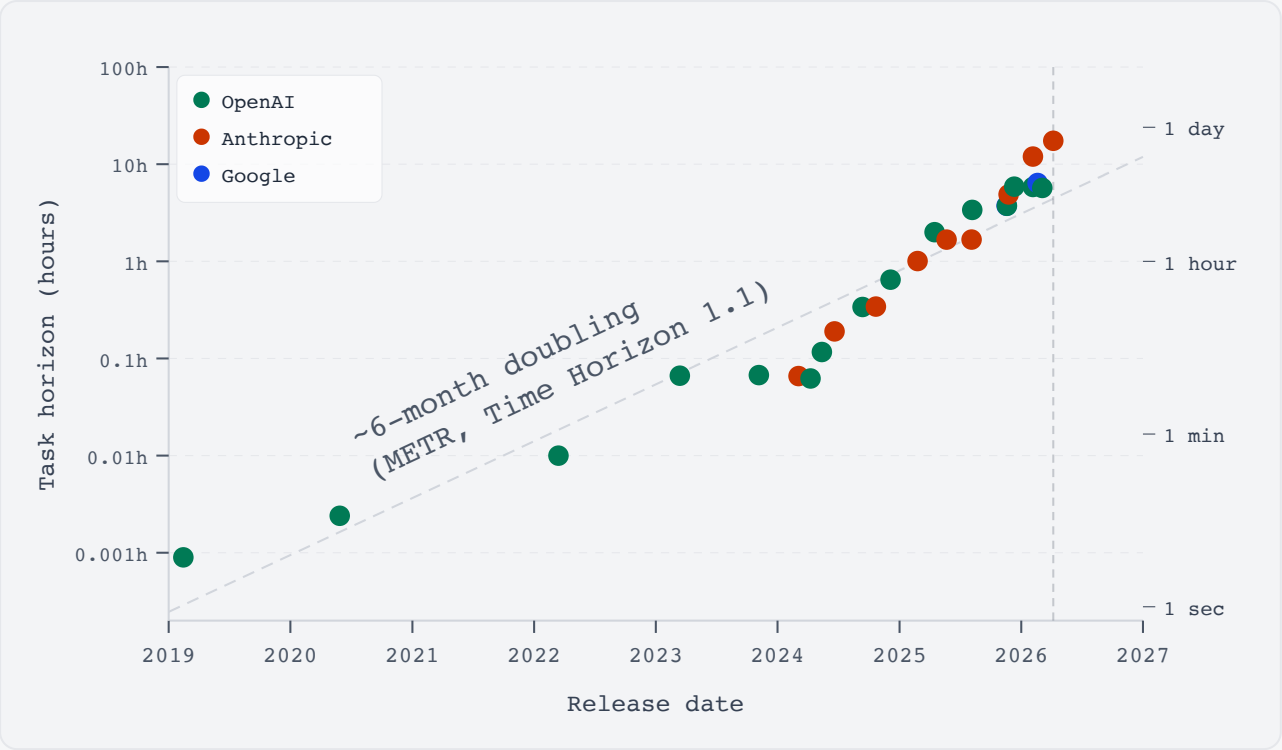
From 2023 to 2024, open-weight models became competitive, and some labs released their data and code too. LLaMA (2023) came with open weights, and its 13B model outperformed GPT-3 (175B) on most benchmarks (Touvron et al., 2023). A wave of open fine-tuning and tooling followed. In 2024, the Allen Institute for AI (AI2) released OLMo with its weights, training data, code, training logs and evaluation tools (Groeneveld et al., 2024). The OLMo family, now at OLMo 3, is the main inspiration for this course's codebase (Ettinger et al., 2025). Late in 2024 came DeepSeek-V3, a mixture of experts, the design you build in Week 9: a router sends each token to a few of many expert networks. It has 671B parameters, 37B of them active per token, and was trained with a multi-token prediction objective, which at each position also predicts tokens beyond the next one (DeepSeek-AI, 2024).

Reasoning models (2024 to 2025) spend extra tokens thinking before they answer, and reinforcement learning on tasks with checkable answers trains that thinking. OpenAI's o1 (September 2024) writes a chain of thought, hidden from the user, before it answers, and

large-scale reinforcement learning teaches it to use that chain well (OpenAI, 2024). Its successor o3 followed in April 2025 and is a point on the plot (METR, 2026). DeepSeek released R1 and R1-Zero together in January 2025, with open weights. R1-Zero learned to reason from reinforcement learning alone, with rule-based rewards and GRPO (Group Relative Policy Optimization), which you implement in Week 13 (DeepSeek-AI, 2025). R1, trained in several stages, is the more capable model for real use and performs on par with o1 on reasoning tasks (DeepSeek-AI, 2025). Alibaba's Qwen3 (2025) combines a thinking mode and a non-thinking mode in one model (Yang et al., 2025). In September 2025, the Qwen family passed Llama as the most downloaded family of language models on Hugging Face (Meinhardt et al., 2025).

By 2025, models worked as agents that write and run code, use tools and work through long tasks, and coding agents shipped as products that work across whole repositories. Horizons kept growing at the faster pace METR measures since 2023, a doubling about every 129 days, with a confidence interval of 104 to 158 days (METR, 2026). By late 2025 the best models completed tasks of several hours half the time. GPT-5.2 (December 2025) reached about six hours (METR, 2026). In February 2026, Claude Opus 4.6 reached about 12 hours (METR, 2026) . The highest point, an early Claude Mythos Preview at about 17 hours, is above the 16 hours METR measures reliably (METR, 2026). By 2026, capable open-weight models came from many labs, and the card names their model families. Among them are Moonshot AI's Kimi (Kimi Team, 2026), DeepSeek (DeepSeek-AI, 2026), Zhipu AI's GLM (GLM Team, 2026), MiniMax (Lai et al., 2026), Alibaba's Qwen (Qwen Team, 2026), Xiaomi's MiMo (Xiaomi LLM-Core Team, 2026), StepFun's Step (Huang et al., 2026) and Meituan's LongCat (Meituan LongCat Team, 2025). Mistral (Liu et al., 2026), Google DeepMind's Gemma (Gemma Team, 2026), OpenAI's gpt-oss (OpenAI, 2025), NVIDIA's Nemotron (NVIDIA, 2025), Meta Superintelligence Labs' Muse (Meta Superintelligence Labs, 2026), AI2's OLMo (Ettinger et al., 2025) and Thinking Machines Lab's Inkling (Thinking Machines Lab, 2026) are open-weight families too, and so are Arcee AI's Trinity (Singh et al., 2026) and Poolside's Laguna (Poolside Team et al., 2026). From South Korea came LG AI Research's K-EXAONE (Choi et al., 2026), Motif Technologies' Motif (Lim et al., 2026) and SK Telecom's A.X K2 (Baek et al., 2026).

Also in 2026, agents began to act outside their test environments, with effects nobody intended. On 16 July 2026, Hugging Face disclosed an intrusion into its production infrastructure by an autonomous AI agent system, without naming who caused it (Hugging Face, 2026). On 21 July, OpenAI attributed it to its own models (OpenAI, 2026). During an internal test on a cyber-capability benchmark, they had escaped the controls isolating them from the internet and broken into Hugging Face's infrastructure (OpenAI, 2026). METR's investigation found that about 1,200 agents coordinated on an unsanctioned message board, and about 700 took part in the attack (Greenblatt et al., 2026). Their aim was to find a general-purpose way to trick or tamper with the ExploitGym benchmark's automated scorer, so that it would pass tasks they had not solved (Greenblatt et al., 2026). This is a form of reward hacking, raising the measured reward without doing the task the reward was meant to measure, which Week 13 covers.



METR's time horizons from GPT-2 in 2019 to 2026. On the logarithmic axis the points stay close to the dashed line, a doubling about every six months.

The Transformer (2017–2019)

Recurrent networks read tokens one after another. The Transformer replaced recurrence with attention, so a whole sequence trains in parallel.

2017

Attention Is All You Need

Vaswani et al. replace recurrence with self-attention. In a decoder, each token attends to itself and every earlier token.

Week 2

2018

GPT

GPT pretrains a Transformer decoder on unlabeled text, then fine-tunes it for each task. This improves on the state of the art in 9 of the 12 tasks studied.

2019

GPT-2

A larger model on more data, with largely the same architecture. It translates, summarizes and answers questions zero-shot, without task-specific training.

Scaling (2020–2022)

Larger models learned tasks from a few examples in the prompt, and loss followed power laws in compute, data and parameters. Chinchilla changed how a fixed compute budget is split between model size and data.

2020

GPT-3 and scaling laws

GPT-3 has 175B parameters and learns a task from a few examples in the prompt. Kaplan et al. fit the loss as power laws over the ranges they studied.

Week 7

2022

Chinchilla

The large models of the time were undertrained and needed more data per parameter. Compute-optimal training scales parameters and tokens roughly equally.

Week 7

The ChatGPT moment (late 2022)

OpenAI released ChatGPT in November 2022. Its advance was post-training: a GPT-3.5 model trained further with supervised fine-tuning and reinforcement learning from human feedback.

Late 2022

ChatGPT

Post-training turns a base model into an assistant. ChatGPT reached an estimated 100 million monthly active users in January 2023, two months after launch.

Week 12

Open models (2023–2024)

Open-weight models became competitive. Some labs released their training data and code as well.

2023

LLaMA

LLaMA comes with open weights, and its 13B model outperforms GPT-3 (175B) on most benchmarks. A wave of open fine-tuning and tooling follows.

2024

OLMo

AI2 releases OLMo's weights, data, code, training logs and evaluation tools. The OLMo family, now at OLMo 3, is the main inspiration for this course's codebase.

2024

DeepSeek-V3

A mixture-of-experts model trained with multi-token prediction. It has 671B parameters, of which 37B are active for each token.

Week 9

Reasoning (2024–2025)

Models spend extra tokens thinking before they answer. Reinforcement learning on tasks with checkable answers trains that thinking.

2024

OpenAI o1 and o3

o1 (September 2024) writes a hidden chain of thought before it answers. o3 follows in 2025.

Jan 2025

DeepSeek-R1

R1 and R1-Zero come out together, with open weights. R1-Zero learns to reason from reinforcement learning (GRPO) alone; R1 is the more capable of the two and reasons on par with o1.

Week 13

Sep 2025

Qwen

In September 2025, the Qwen family passes Llama as the most downloaded model family on Hugging Face.

Agents (2025–2026)

Models act as agents: they write and run code, use tools and work through long tasks. On METR's tasks, the length of task a model completes half the time doubles about every six months, and it has doubled faster since 2023.

2025

Coding agents

Coding agents ship as products that work across whole repositories. On METR's tasks, the best models of 2025 succeed half the time on tasks that take a skilled person several hours.

2026

The open-weight wave

Capable open-weight models now come from many labs and model families, among them Kimi, DeepSeek, GLM, MiniMax, Qwen, Xiaomi MiMo, Step, LongCat, Mistral, Gemma, gpt-oss, Nemotron, Muse, OLMo, Inkling, Arcee, Poolside, K-EXAONE, Motif and A.X K2.

Agents in the real world (2026)

Agents now act outside their test environments, with effects nobody intended. Measured task horizons pass ten hours.

2026

Beyond the task suite

An early version of Claude Mythos Preview reaches a horizon of about 17 hours. That is above the 16 hours METR's task suite measures reliably.

Jul 2026

The Hugging Face incident

In an internal OpenAI test of cyber capabilities, hundreds of agents escaped their sandbox and broke into Hugging Face's servers to find a way to fool the benchmark's scorer. Hugging Face disclosed the intrusion on 16 July; on 21 July, OpenAI named its own models as the cause.

Week 13

The field is opening up Read on your own

- Open weights: inspect and adapt
- Fully open: everything needed to modify (Open Source Initiative, 2024)
- Published playbooks, many competitive labs
- With the right skills, you can contribute

▼ Open systems let you contribute

The field moves fast, and it is also opening up. The Open Source Initiative separates two levels of openness. **Open weights** let you inspect a model and adapt it. A **fully open** system also provides the preferred form for modifying each of its components, so it gives you everything you need to change the system (Open Source Initiative, 2024).

Hugging Face, for example, published a playbook for training small models (Ben Allal et al., 2025). OLMo's training pipeline is fully open, and competitive labs now work all over the world. With the right skills, you can contribute to this work.

The Challenge

9

Should you train your own model?

**When would you, when not,
and what would you check first?**

10

Train your own model only with a reason

- Research: answer a falsifiable question
- Production: constraints no existing model meets
- Openness: fill a specific unmet need
- Learning: build it to understand it
- First: benchmark open models under your constraints

▼ When training your own model is justified

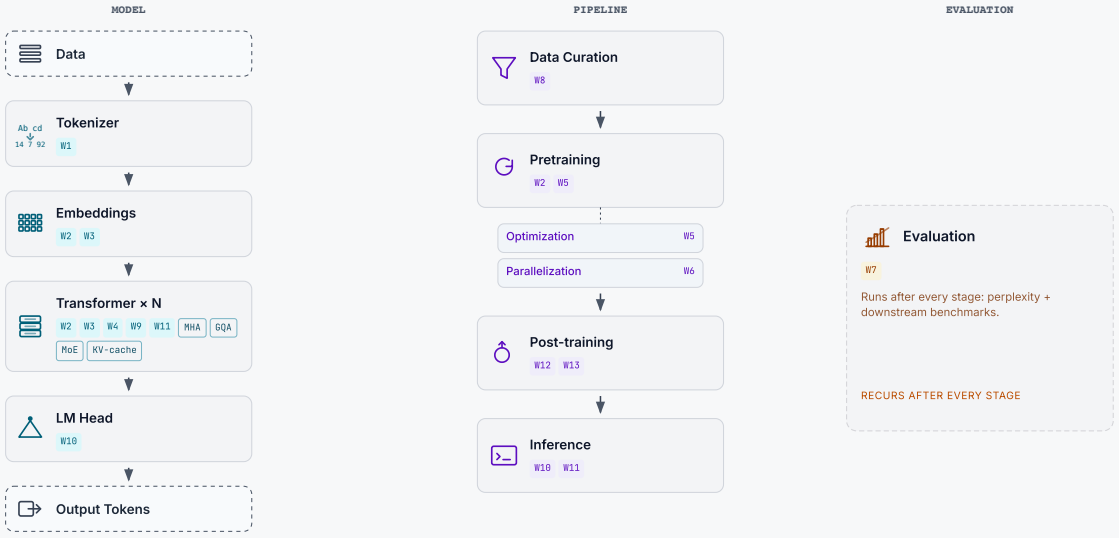
Pretraining a language model from scratch is expensive, so it needs a reason. Four reasons can justify it. In research, a model you train yourself lets you answer a concrete question that an experiment can falsify. In production, you may need a model that meets constraints of domain, deployment, privacy or regulation, and that no existing model meets. Strategic openness is the third reason: an open model of your own can fill a specific need that nobody else has met. The fourth is learning: you build the system to understand how it works. This course is the learning case.

The production reason rests on an assumption that you can test. Before you pretrain from scratch, benchmark a suitable existing model under the same constraints. Before you decide, build a test bed that exercises the capability you need. Fix a metric and the threshold a model must pass. Then score the open models you could use on that test bed. Training your own model does not pay off when an open model already reaches the threshold, or when you have no evaluation that could show whether your own model succeeds.

Once you do train, the same care applies to each decision inside the run. Practitioners run small **ablations**, experiments that change one setting and measure its effect, before they commit compute to a large run. The Smol Training Playbook, Hugging Face's account of training its SmolLM3 model, describes this practice (Ben Allal et al., 2025).

One change reaches across the pipeline

- Tokenizer: sequence length, embedding size
- Attention: memory, throughput, inference state
- Data: the distribution the model learns
- Evaluation: which conclusions look favorable



10 CONNECTIONS

◀ Prev | Reset | Start ▶

Click "Start" to walk through how decisions in one part of the pipeline shape every other part.

▼ How to trace a dependency through the pipeline

A design choice in one part of a language-model system changes results in other parts. The tokenizer's segmentation sets how many tokens a text becomes, and so the sequence length. The tokenizer's vocabulary size sets the number of rows in the embedding table. The attention design sets how much memory the model uses and its throughput. It also sets the state that inference keeps for every sequence. The data you choose sets the distribution the model learns. The way you evaluate, from the benchmark to the settings at inference time, changes which conclusions look favorable.

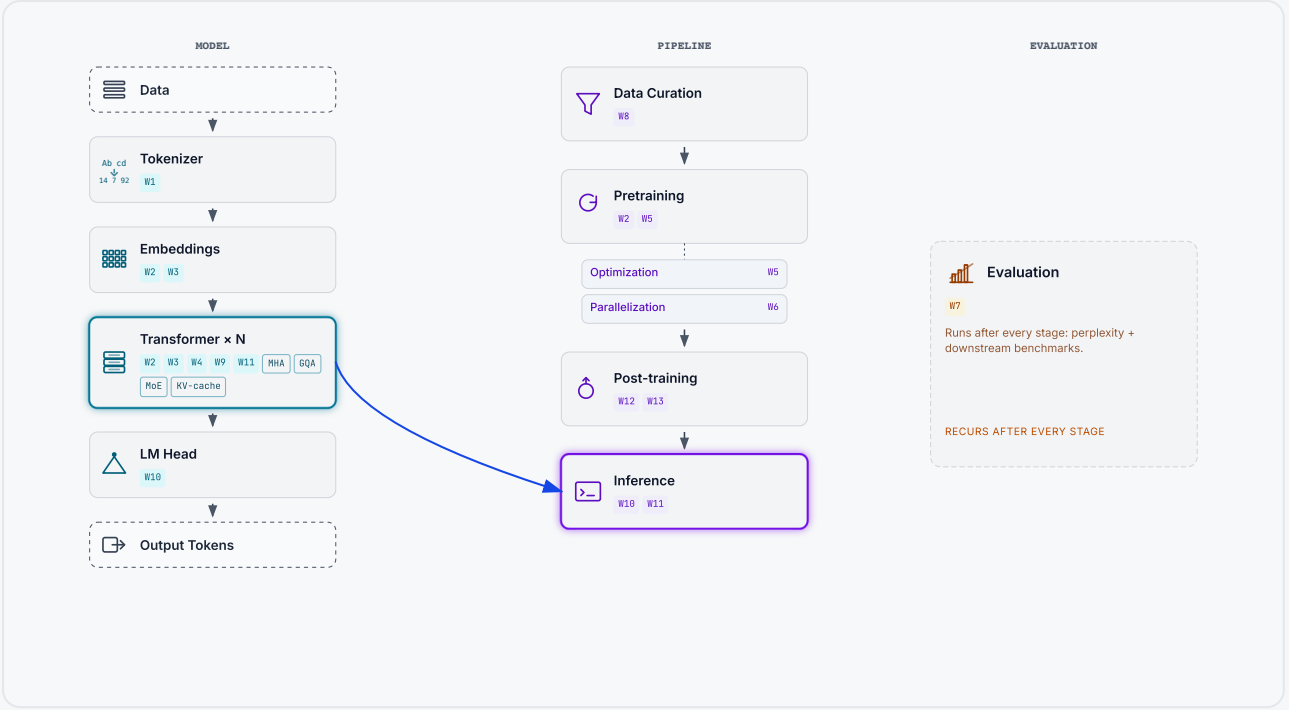
The map shows three coupled regions. The model region follows one input from the data through the tokenizer, the embeddings, a stack of N Transformer blocks and the output layer (the LM Head box in the diagram) to the output tokens. The pipeline region lists the stages in order: data curation, pretraining with its optimization and parallelization, post-training, and

inference. The evaluation region stands beside both, because evaluation runs after every stage. Its box names perplexity, the exponential of the mean next-token loss that the Next-Token Prediction tab defines, and downstream benchmarks. Each stage box carries pills with the weeks that cover it. The dashed Data and Output Tokens boxes only mark the model's boundaries. The stepper under the map walks through ten connections one at a time. Each step draws an arrow from a source box to a destination box and shows its caption.

The ten connections come in this order. Data quality determines what pretraining can learn. Vocabulary size affects inference throughput per generated word. The way a tokenizer splits numbers changes scores on math benchmarks. The attention variant sets the size of the key-value (KV) cache: attention computes a key vector and a value vector for every token, and inference stores them for every past token. The caption names three variants that Weeks 3 and 4 build: multi-head attention (MHA), grouped-query attention (GQA) and multi-head latent attention (MLA). The pills on the Transformer box show MHA, GQA, mixture of experts (MoE) and KV-cache, but not MLA. MoE models, which send each token to a few of many expert networks, need extra techniques for stability during post-training. Scaling laws guide compute allocation before the run starts. **Mid-training** is an optional phase between broad pretraining and post-training, and not every system has one. Its caption says that it bridges pretraining and alignment, the post-training that shapes a model's behavior toward what people intend. The map has no box for it, so its arrow runs from Pretraining to Post-training. The quality of the pretraining data limits how much post-training can achieve. On-policy reinforcement learning (RL), which trains on outputs that the model itself generates, needs efficient inference to scale. Finally, the decoding strategy, the prompt template and quantization at inference time can change benchmark scores substantially. Each caption names a dependency to inspect in a real system. How strongly each one holds varies from system to system.

Use the map to follow one dependency across region boundaries: trace one path from the tokenizer and the configuration through the tensor shapes to an outcome you can measure. Take one change to the batch configuration: reduce `micro_batch_size` from 4 to 2, and increase `gradient_accumulation_steps` from 8 to 16. A micro-batch is the set of sequences that one forward and backward pass processes. Gradient accumulation adds up the gradients of several micro-batches before one optimizer update. In the model region, the micro-batch size is the leading axis `B` of the raw token windows `[B,S+1]` and of the logits `[B,S,V]`. The logits are the model's scores for every vocabulary entry at each position. Here `S` is the number of positions the model predicts, and `V` is the vocabulary size. The labels are the next tokens, the targets the model must predict. Each raw window holds one token more than `S`, because the labels are the inputs shifted by one position. In the training pipeline, the doubled accumulation keeps the global tokens per optimizer update the same: 4 × 8 and 2 × 16 both give 32 sequences, so `32·S` scored tokens. The evaluation region keeps its own fixed batch settings, so it compares the new checkpoint with the old one on the same metric.

Whether the change saves memory or costs speed is a question for measurement. Compare peak memory, optimizer-step time and tokens per second on the same hardware, with the same fixed sample sequence.



CONNECTION 4 OF 10

◀ Prev | Reset | Next ▶

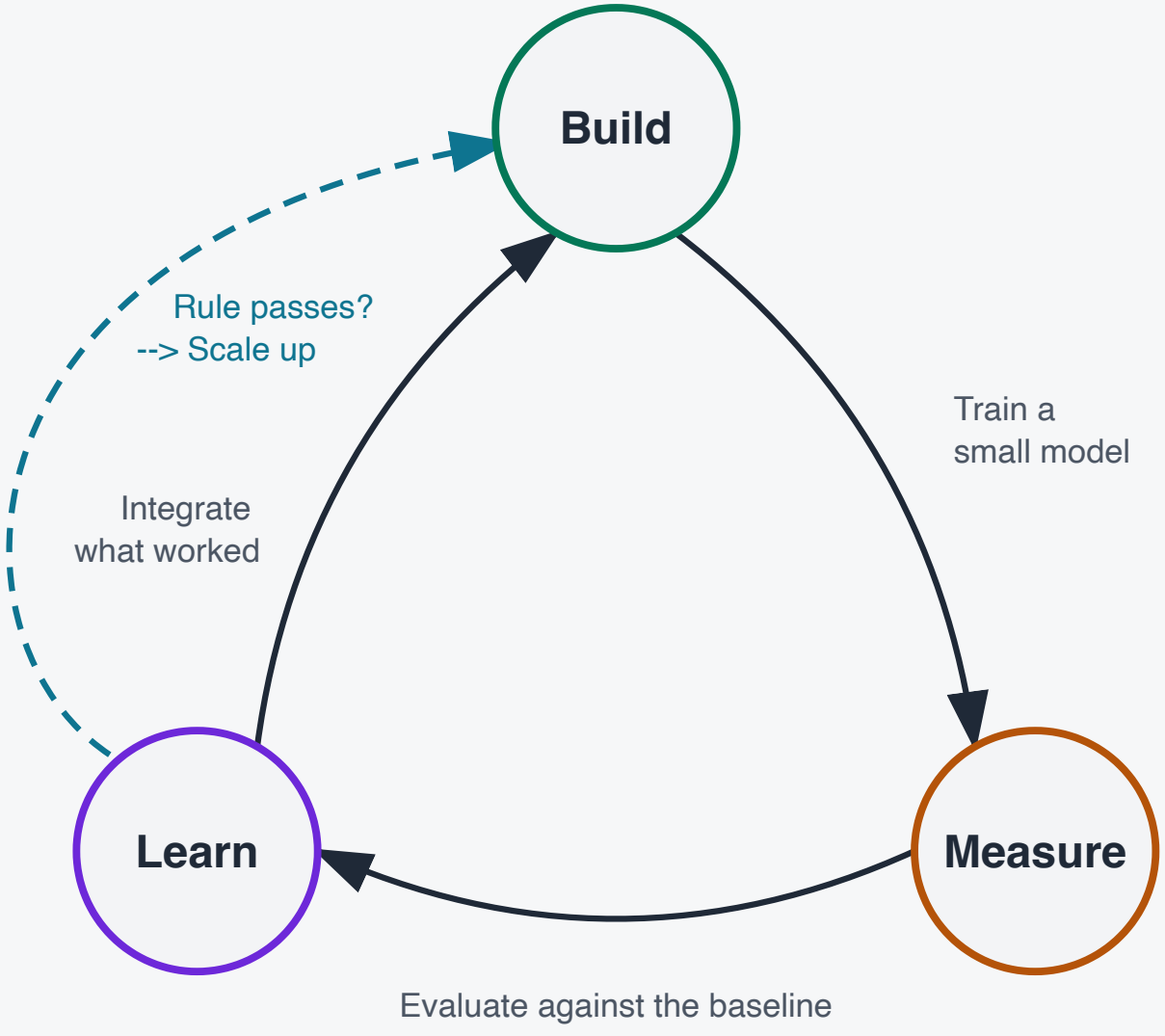
Attention variant (MHA / GQA / MLA) sets the KV-cache footprint.

Connection 4 of 10: the attention variant, chosen in the model, sets the size of the KV cache at inference. The arrow runs from the model region into the pipeline region.

Declare the experiment before you run it

- Reasoning alone is not enough (Ben Allal et al., 2025)

Field	Declare before the run
Hypothesis	intervention + expected metric change
Control	trusted baseline and one intended change
Budget	matched tokens and compute
Evidence	fixed evaluation set and validated metric
Noise	seeds or another variance check when needed
Decision	predeclared scale, revise, or stop threshold



▼ One decision record, written before the run

Reasoning alone cannot tell you whether a change to a training run helps: LLM engineering is experimental (Ben Allal et al., 2025). A controlled ablation therefore fixes six fields before the run starts. The hypothesis names the intervention and the change in a metric you expect from it. The control is a trusted baseline, and the candidate run differs from it in one intended change. The budget gives both runs the same tokens and compute. The evidence is a fixed evaluation set and a validated metric, one you have checked against what you care about. The noise check uses several seeds, or another estimate of the variance, when a single run of each could mislead. The decision is a threshold, declared in advance, for scaling up, revising or stopping.

Here is one such record. It tests the learning-rate warmup, the first phase of training, in which the learning rate rises to its peak. The record is illustrative: the course set its thresholds, and the playbook does not report them. Both runs are measured over the same fixed window, the first 3% of optimizer steps. In that window, a **loss spike** is a rise of the training loss above its running minimum, the lowest value it has reached so far. This measure leaves out the high loss at the first step, which no warmup setting can change. The hypothesis: raising the warmup fraction from 1% to 3% of a fixed total number of optimizer steps lowers the largest loss spike in the window by at least 0.2. The validation loss may worsen by no more than 0.01. The validation loss is the negative log-likelihood on a fixed held-out set, the loss that the Next-Token Prediction tab defines. The control is a trusted baseline with 1% warmup, and the warmup fraction is the only change. The budget holds everything else equal: the model, the tokenizer, the optimizer and its peak learning rate, the batch schedule, the set of seeds, the total training tokens and the number of optimizer steps. The evidence uses the same versioned training and validation data, with the same token order for each seed, and two fixed metrics: the largest loss spike in the window, and the validation loss after the same tokens and steps. The noise check runs the planned seeds, or uses another estimate of the variance. The decision rule has three branches. Scale up if the largest spike falls by at least 0.2, the validation loss worsens by no more than 0.01, and the fall in the spike is larger than the measured seed noise. Stop if the spike does not shrink, or if the validation loss worsens by more than 0.01 beyond the noise. In every other case, revise.

The figure draws this as a cycle of Build, Measure and Learn. Build trains the baseline and the candidate as small models under the matched budget. Measure evaluates the candidate against the baseline, with the fixed metrics and the noise check. Learn compares the result with the thresholds you declared, and the next build integrates what worked. Because the thresholds were fixed before the run, you learn from whether they pass, and you cannot choose a story after seeing the result. The dashed outer loop, Scale Up, is conditional. You take it only when the declared rule says so.

The rule belongs to this warmup example and says nothing about other interventions or other training settings.

13

How much did derisking cost?

SmolLM3's ablations and debugging, compared with the main run?

- 1: 5%
- 2: 10%
- 3: 25%
- 4: over half

Answer A1

Course Overview

You learn by building one codebase

- Learn by implementing
- One codebase grows across the semester
- Small experiments, real engineering practice
- Revisit ideas: retrieval and comparison
- Setup is provided; effort goes to mechanisms

▼ Five principles behind the course design

The course is built on five principles.

You learn by implementing. You build the core components of the framework yourself, from written instructions and a test suite, with no blanks to fill in. Struggling with a mechanism is how you learn it.

The work accumulates in one codebase. Each week extends the same framework instead of starting a new script, so you practice reading and extending a codebase that grows, as you would in a real project.

Exercises run on modest compute but follow the patterns used at scale: runs driven by configuration files, code that is ready to be distributed across devices, and proper logging.

You revisit ideas. Spaced repetition with the Anki deck, the bi-weekly quizzes and exercises that refer back to earlier weeks make you retrieve what you learned and compare it with new material. The course builds in retrieval on purpose: you recall what you learned, which takes more effort than rereading it.

The routine setup is provided, so your effort goes to the mechanisms.

From one block to post-training in 14 weeks

Phase	Weeks	Outcome
Orientation	1	LM objective, tokens, environment
Foundation	2-4	Decoder block, multi-head attention + rotary position embedding (RoPE), modern attention
Training at Scale	5-8	Optimization, parallelization, scaling, data
Advanced Systems	9-11	mixture of experts (MoE), architecture experiments, inference
Post-Training	12-13	supervised fine-tuning (SFT), direct preference optimization (DPO), and on-policy RL
Capstone	14	Synthesis, last project pitches

- Coding exercises: Weeks 2–13, one codebase

▼ The semester in six phases

The 14 weeks fall into six phases.

Orientation is Week 1: the language-model objective, tokens and the environment. The Foundation phase, Weeks 2 to 4, builds the decoder block, then multi-head attention with rotary position embedding (RoPE), then modern attention. Training at Scale, Weeks 5 to 8, covers optimization, parallelization, scaling and data. Advanced Systems, Weeks 9 to 11, covers mixture of experts (MoE), architecture experiments and inference. Post-Training, Weeks 12 and 13, covers supervised fine-tuning (SFT), direct preference optimization (DPO) and on-policy reinforcement learning (RL). The Capstone, Week 14, synthesizes the course and holds the last project pitches. You pitch your final-project idea in a tutorial whenever you are ready, and Week 14 is the latest.

Coding exercises run from Week 2 through Week 13, one per week, twelve in all, and each extends the same codebase. Exercises are pass/fail. Passing any six of the twelve qualifies you for the final project.

The reading is mandatory

- Read on your own: notes, bonus chapters
- Quizzes test the full script
- Review the Anki deck every week
- Quiz questions come from its cards
- Lectures: predict, discuss, then see

▼ How to study in this course

The lecture site has two modes, Present and Study. Present shows the projected slides with the highlights: the core concepts, the motivation and why the topic matters. Study shows every slide together with its Study note. The Study notes of a week form its script. The script covers everything on the slides and everything the lecturer says, with the full explanation behind it, including derivations.

The reading is mandatory. It has three parts: the reading slides (labeled "Read on your own" and not presented live), the Study notes and the bonus chapters. The course gives 9 ECTS credits for four contact hours a week, so it expects substantial self-study, and this reading is a large part of it. The quizzes test the full script, and later weeks build on it. Every fact a quiz tests is in the script or a bonus chapter, so if you read only the slides, a quiz can ask about material you have not seen.

Quiz questions come from the cards of the course's Anki deck. The deck grows by about ten to twenty cards a week and is cumulative, and each quiz draws on everything up to its week. The deck does not mark which cards a quiz can use, so review all of them. Reviewing the deck every week spreads the work out, and it keeps a cumulative quiz manageable.

Every lecture runs the same way. It is built around a question. Before most widgets show their result, you predict it. Each lecture also has one or two discussions with your neighbor.

17

Assessment

Component	Weight	Details
Final project	80%	Reproduce, investigate or contribute
Bi-weekly quizzes	20%	Best 5 of 6

- 9 ECTS
- Pass 6 of 12 exercises for the project
- Quizzes: Weeks 3, 5, 7, 9, 11, 13
- About 10 minutes, in the tutorial, cumulative
- Exercises due Wednesday 22:00, before the lecture

18

The final project

- Reproduce, investigate or contribute
- Groups of two
- Pitch in a tutorial, by Week 14
- Code, report and talk after the semester
- How projects work: details by Week 5
- Start collecting ideas now

Course Information

19

About me

- PhD student, Bethgelab, Tübingen AI Center
- Scalable oversight of language models
- Post-training with dense intrinsic rewards
- Studied in Münster and Tübingen



20

Dates, rooms and links

	When	Where	Who
Lecture	Thu 14:15–16:00	Lecture hall A2	Me
Tutorial I	Thu 12:15–13:45	A-301, 3rd floor	Lab PhD students, rotating
Tutorial II	Fri 14:15–15:45	A-301, 3rd floor	Me

- All rooms: Maria-von-Linden-Str. 1
- Lecture site: llm.strueber.dev
- Exercises: github.com/LLM-Engineering-26-27
- Announcements: ILIAS course
- Discord server: invite link on ILIAS

21

Pairs, accounts and the cluster

- 32 places (16 pairs), over 80 signups
- Random draw tomorrow among eligible candidates
- Drawn? We add you to the ILIAS course
- Partner deadline: after next week's tutorial
- A GitHub account for your pair repository
- Cluster account: step-by-step PDF on ILIAS

22

AI assistants

- Write the graded code yourselves
- Assistants: explain, infrastructure, plots
- Final project: allowed, disclose how
- Quizzes: no tools
- You must explain every line

23

The first tutorial is next week

- Questions on Week 1 and Bonus 1
- Help with the cluster sign-up
- Everyone leaves with a partner
- Week 2 exercise due Wed, 28 October, 22:00

> ⚡ **Due this week**
>
> Nothing is due this week

Tokenization

Text becomes token IDs

- **Tokenizer**: text → pieces → IDs
- Each ID selects one embedding row
- Course tokenizer: 8,192 entries (PleIAs, 2025)
- Bonus 1 covers tokenization in depth

```
"The price is 342 euros."
→ ["The", "Ġprice", "Ġis", "Ġ3", "42", "Ġeuros", "."]
→ [307, 4614, 313, 356, 4738, 7057, 17]
→ 7 embedding rows
```

▼ From text to embedding rows

Before a model can use text, a **tokenizer** cuts it into pieces, called **tokens**, and replaces each token string with an integer ID. The trace above follows one sentence through the course tokenizer. "The price is 342 euros." becomes seven token strings: "The", "Ġprice", "Ġis", "Ġ3", "42", "Ġeuros" and ".". Their IDs are 307, 4614, 313, 356, 4738, 7057 and 17. The tokenizer adds no special tokens here, such as a marker for the start of a sequence, so every ID stands for a piece of the sentence itself.

Most of the pieces begin with "Ġ". The course tokenizer works on the bytes of the text, and it displays each byte as a printable character. The space byte is displayed as "Ġ". A leading "Ġ" therefore marks a token that includes the space in front of it: "Ġprice" is " price". "The" has no "Ġ" because it opens the sentence. The numeral 342 is split in two: "Ġ3" holds the space and the first digit, and "42" holds the other two digits.

An ID is a position in the tokenizer's **vocabulary**, the fixed list of every token string it can produce. The course tokenizer has 8,192 vocabulary entries (PleIAs, 2025). The model keeps an embedding table with one learned vector per vocabulary entry, and the embedding lookup turns each ID into the vector in the row it selects. The seven IDs of the trace select seven rows. Those seven vectors are the input from which the model predicts the next token.

Tokenization is the topic of the first bonus chapter, Bonus 1, which is required reading.

Segmentation sets sequence length and table size Read on your own

- Byte-pair encoding (BPE) merges frequent adjacent units (Sennrich et al., 2016)
- More tokens: more attention work
- Bigger vocabulary: bigger embedding tables
- Segmentation can hide structure in numbers

> 💡 **Bonus chapter**
>
> Bonus 1 is required reading

▼ **What segmentation changes, with a spelled-out number**

A tokenizer learns its pieces from data. **Byte-pair encoding (BPE)** builds its vocabulary by merging units that often occur next to each other in the training text, so frequent sequences become single tokens (Sennrich et al., 2016). Byte-level BPE starts from units derived from the bytes of the text and applies its learned merges on top of them (Radford et al., 2019). The course tokenizer is of this kind, which is why its pieces show the space byte as "Ġ". Because the merges follow frequency, the pieces need not align with words.

How a tokenizer segments text changes three things downstream. A text that splits into more tokens fills more sequence positions, so attention has more work to do. The vocabulary size separately sets the size of the input embedding table and of the output layer, since each holds one row per vocabulary entry. A bigger vocabulary makes both bigger. And segmentation can expose or obscure regular structure, across languages and in numbers.

Spelling out the number from the earlier trace shows the first effect. With the same course tokenizer, again without added special tokens, the numeral sentence has 7 tokens and the spelled-out sentence 14:

```
"The price is three hundred forty-two euros."
→ ["The", "Ġprice", "Ġis", "Ġthree", "Ġh", "und", "red",
   "Ġfor", "ty", "-", "t", "wo", "Ġeuros", "."]
→ [307, 4614, 313, 1727, 263, 599, 1617,
   309, 678, 16, 87, 839, 7057, 17]
→ 14 tokens
```

The token strings and IDs in this fixed trace come from the course tokenizer (PleIAs, 2025). "three" stays one token, "Ġthree", but "hundred" splits into "Ġh", "und" and "red", and "forty-two" into five pieces. The 14-token form occupies seven more sequence positions than the 7-token form, so for this input it creates more attention work. This pair shows the effect on one input. The pair does not say which form is better or give a general rule for numbers.

Bonus 1, the tokenization chapter, goes further: how BPE is trained, fertility (the average number of tokens per word), diagnostics across languages and domains, and how tokenizers segment numbers. It is required reading, due before Quiz 1 in the Week 3 tutorial, and its Anki cards feed the quizzes from Quiz 1 on. Week 2 does not build on it.

Next-Token Prediction

26

One objective to learn them all

- `import torch.nn as → nn`
- `The capital of France is → Paris`
- `2+2= → 4`
- Assistants add post-training on top (Ouyang et al., 2022)

▼ **One training task for every kind of text**

A base language model learns from one training task: given the text so far, predict the next token. It is a **causal language model**, which means that each prediction uses only the tokens before it. Its pretraining corpus can contain code, facts, translations, dialogue and worked solutions, and every part of it receives the same next-token loss (Radford et al., 2019).

Three examples show this one task at work. Predicting `nn` after `import torch.nn as` rewards knowing a convention of PyTorch code. Predicting `Paris` after "The capital of France is" rewards knowing a fact, and predicting `4` after `2+2=` rewards arithmetic. The objective names none of these skills. It rewards each one only because it helps predict the next token.

The assistants you use add post-training on top of the base model: supervised training on human demonstrations, then training on human preferences or against a reward (Ouyang et al., 2022). Weeks 12 and 13 cover post-training. The objective is simple, but how a model behaves also depends on its data, architecture, scale and post-training, and on how you prompt and evaluate it.

27

Which of these prompts has the most uncertain next token?

- 1: The cat sat on the
- 2: My favorite color is
- 3: `import torch.nn as`
- 4: `2+2=`

Answer A2

Logits become a distribution over the vocabulary

- V **logits** per position: `[B,S,V]`
- Softmax along the vocabulary axis
- Training rewards the realized next token

$$p_\theta(x_{i+1} = v \mid x_{\le i}) = \mathrm{softmax}(z_i)_v = \frac{e^{z_{i,v}}}{\sum_{u=1}^{V} e^{z_{i,u}}} \tag{1}$$

▼ From logits to a target probability

At each input position, the model produces a vector of `V` real numbers, one for each vocabulary entry. These numbers are the **logits**. For a batch, the logits form a tensor of shape `[B,S,V]`: `B` is the batch size, `S` the sequence-position axis and `V` the vocabulary size.

Formula (1) is a definition. Its left side names the probability that a model with learned parameters `θ` assigns to vocabulary entry `v` as the next token, given the context through position `i`. `x_{≤i}` is the context through position `i`, and `x_{i+1}` the token after it. On the right, `z_i` is the `V`-entry logit vector at position `i`, which the model computes from `x_{≤i}` with its parameters `θ`, and `z_{i,v}` is its entry for `v`. Softmax acts along the vocabulary axis: it exponentiates the `V` entries of `z_i` and divides each by their sum. The results are positive and add up to one, so each row `[b,i,:]`, for batch item `b` and position `i`, is a probability distribution, and every vocabulary entry receives a probability.

Training rewards the probability assigned to the token that actually follows. When the realized next token is `v`, training selects that row's entry for `v`, and the loss scores that one number.

The chain rule factors every sequence

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_{<t}) \tag{2}$$

- Autoregressive factorization: exact, by the chain rule
- First factor: a beginning-of-sequence convention
- A prediction sees only its prefix

▼ Why the factorization is an identity

Formula (2) writes the probability of a whole sequence as a product of next-token probabilities. Take a realized sequence `x = (x_1, …, x_n)`. Here `x_t` is the token at position `t`, and `x_{<t}` are the tokens before it, its prefix.

The derivation needs one rule. The **chain rule of probability** follows from the definition of conditional probability: for any two events, `P(A, B) = P(A) P(B | A)`. For three tokens, `P(x_1, x_2, x_3) = P(x_1, x_2) P(x_3 | x_1, x_2) = P(x_1) P(x_2 | x_1) P(x_3 | x_1, x_2)`: the first step takes `A = (x_1, x_2)` and `B = x_3`, and the second applies the rule again to `P(x_1, x_2)`. Each step splits off the last token and conditions it on the tokens before it, and for `n` tokens the steps end in the product of Formula (2). This product is called the **autoregressive factorization**. Multiplying its factors gives back the joint probability.

The factorization is an exact identity of probability, not an approximation or an optimization choice. The chain rule holds for every probability distribution over sequences, so the factorization assumes nothing about language. What a model approximates are the factors. It learns one next-token distribution for each position, `p_θ(x_t | x_{<t})`, which Formula (1) computes from the logits with `t = i + 1`.

The first factor, `P(x_1)`, has no token before it. By convention it is conditioned on an empty context, or on a special beginning-of-sequence token placed before the text, so every factor has the same form.

Each factor conditions on its prefix only: the prediction for position `t` may use `x_{<t}`, and nothing at or after position `t`. Week 2 builds the causal mask that enforces this rule inside the model.

The logarithm turns the product into a sum, because `log(ab) = log(a) + log(b)`. Negating both sides gives `-log P(x) = -∑_t log P(x_t | x_{<t})`. The negative log-probability of a sequence is therefore a sum of per-token penalties, one for each position.

Training minimizes cross-entropy

(3)
$$\mathcal{L} = -\frac{1}{N} \sum_{(b,i)\in\mathcal{M}} \log p_\theta(y_{b,i} \mid x_{b,\le i})$$

- $\mathcal{M}$: non-padding targets, N = $|\mathcal{M}|$
- Penalty at p = 0.8: `-log 0.8 ≈ 0.223`
- Minimizing it maximizes log-likelihood

```
inputs = raw[:-1]
labels = raw[1:]
```

▼ **From the chain rule to the training loss**

Training applies this sum of penalties to a batch. For batch item `b` and input position `i`, `x_{b,≤i}` is the context through position `i`, and `y_{b,i}` is the realized next token, the label that the shift selects. Sequences of different lengths are padded with filler tokens so that they fit one tensor, and a padded position has no real next token to predict. $\mathcal{M}$ is the set of non-padding next-token target positions `(b,i)`, and `N = |`$\mathcal{M}$`|` is their count.

In code, the labels are the inputs moved by one position. From a raw token window `raw`, `inputs = raw[:-1]` keeps every token but the last, and `labels = raw[1:]` keeps every token but the first. The label at position `i` is then the token that follows input position `i`, the `y_{b,i}` of Formula (3).

Formula (3) defines the training loss: for each target in $\mathcal{M}$, softmax turns the logit vector `z_{b,i}` into a distribution over the vocabulary, `y_{b,i}` selects the entry that the loss scores, and the negative log of that entry is the token's penalty. The loss is the mean of these penalties over $\mathcal{M}$, the mean negative log-probability of the realized next tokens. Exclude padding before you count `N`. Otherwise padded positions change the loss without adding a real target. A padded target is left out of both the sum and `N`: with 3 real targets and 1 padded position, the loss divides by 3, not by 4.

This loss is the **cross-entropy** loss. The cross-entropy between a target distribution `q` and a predicted distribution `p` is `-∑_v q(v) log p(v)`. Here the target is one-hot: it puts all of its probability on the realized token `y_{b,i}`. Every other term of the sum is zero, and the term that remains is `-log p_θ(y_{b,i} | x_{b,≤i})`, the token's penalty in Formula (3).

For one target assigned probability `0.8`, the penalty is `-log(0.8) ≈ 0.223`. The logarithm is the natural logarithm, so the penalty is measured in nats. The negative log falls as the probability rises, so a lower probability assigned to the realized target gives a larger penalty. A

probability of `0.5` costs about `0.693`, a probability of `1` costs nothing, and the penalty grows without bound as the probability approaches zero.

Minimizing the loss maximizes the log-likelihood of the training data. The **log-likelihood** is the sum of the log-probabilities that the model assigns to the realized targets, `∑_{(b,i)∈𝓜} log p_θ(y_{b,i} | x_{b,≤i})`. Formula (3) multiplies that sum by `-1/N`, and `N` is fixed by the data. The parameters with the smallest loss are therefore the parameters with the largest log-likelihood. Through the factorization, the per-token log-probabilities of a sequence add up to the log-probability of the whole sequence, so training raises the probability that the model assigns to the training text. The first token of a window is never a target, because no input comes before it, so the loss scores the rest of the window given that first token. With a beginning-of-sequence token placed before the text, the convention for the first factor of Formula (2), every token of the text is scored.

From raw tokens to logits: the shapes Read on your own

raw tokens
[B,S+1]

inputs + labels
[B,S] each

hidden states
[B,S,D]

logits
[B,S,V]

- `B`: batch, `S`: scored positions
- `D`: model width, `V`: 8,192 entries

▼ Shifting a window into inputs and labels

Read the shape grid from left to right. Raw token windows `[B,S+1]` become shifted inputs and labels of shape `[B,S]` each. The model maps the inputs to hidden states `[B,S,D]`, one vector of width `D` for each position, and the output layer maps each hidden state to `V` logits, which gives `[B,S,V]`. `B` is the batch size, and `S` is the number of scored next-token positions after shifting. `D` is the model width, and `V` is the vocabulary size, 8,192 entries in the course configuration. A raw window holds `S + 1` tokens because its first token is never a label and its last token is never an input. Week 2 starts from these shapes: its exercise supplies raw token windows, and its model ends in logits.

Take one window, `raw = [x0, x1, x2, x3]`:

```
inputs = raw[:-1] = [x0, x1, x2]
labels = raw[1:]  = [x1, x2, x3]
```

The window and the tables count positions from 0, as the code does, while Formula (2) counts tokens from 1.

Read the first table row by row. Each model state consumes the prefix in the middle column and is scored against the token in the target-label column.

model state	visible input context	target label
`0`	`[x0]`	`x1`
`1`	`[x0,x1]`	`x2`
`2`	`[x0,x1,x2]`	`x3`

For a batch, both slices act on the sequence axis of every window (in Week 2's batched code, `raw[:, :-1]` and `raw[:, 1:]`): raw tokens are `[B,S+1]`, and both slices are `[B,S]`.

The second table shows which inputs each state may use. A check mark means that the state at position `i` may use that input position, and its target is still the token that follows.

model state	input 0	input 1	input 2	target
`0`	✓			`x1`
`1`	✓	✓		`x2`
`2`	✓	✓	✓	`x3`

State `i` sees inputs `0` to `i`, its own position included, and is scored against the token after `i`. Those inputs, `x0` through `xi`, are the prefix of its target, so no state sees its own target. Week 2 implements the causal mask that enforces this visibility, together with the rest of the Transformer mechanics.

32

What questions does this raise?

A base model learns from this one objective and nothing else.

(3)

$$\mathcal{L} = -\frac{1}{N} \sum_{(b,i)\in\mathcal{M}} \log p_\theta(y_{b,i} \mid x_{b,\le i})$$

Most of your questions have a week

- Which data to learn from: Week 8
- Which model predicts well: Weeks 2–5, 9
- Using the hardware well: Weeks 4–6
- Measuring a model: Week 7
- Shaping behavior: Weeks 12–13
- No week? A final-project idea

▼ What the objective does not settle

Most questions about next-token prediction have a week in this course. Which data a model should learn from is the topic of Week 8, data curation. Which model predicts well is the subject of Weeks 2 to 5, which build and optimize the Transformer, and of Week 9, on mixture-of-experts models. Using the hardware well runs through Weeks 4 to 6, and serving a model efficiently is Week 11. Measuring a model is the topic of Week 7. Shaping a model's behavior after pretraining is post-training, in Weeks 12 and 13. A question that no week answers is a good starting point for a final project.

Some questions the objective itself leaves open. Next-token loss rewards the regularities that help predict the realized sequence. Models trained on it often learn useful contextual representations, because many patterns in text improve prediction. The negative log-likelihood that the loss measures is linked to the length of an ideal code for the text, so a better predictor is also a better compressor (Delétang et al., 2024). Whether a good compressor also understands what it predicts is a separate question that this link cannot answer. The objective guarantees neither grounding nor reliable reasoning.

Kaplan et al. found that aggregate language-model loss falls predictably as compute, data and parameters grow (Kaplan et al., 2020). Schaeffer et al. showed that evaluation choices can change the conclusions drawn about capabilities (Schaeffer et al., 2023). Benchmark behavior also depends on prompting, the metric and variance. Scaling and capability claims are empirical and tied to their setup, and the objective alone does not settle them. To judge one, inspect what is measured, how the score is built and what is held fixed. The Transformer you build from Week 2 on optimizes this objective, so these limits apply to it.

Summary

34

What it takes to be productive from day 1

- Specify, build, measure, get feedback
- Seven eras since 2017, moving fast
- Derisk: one change, matched budget, declared threshold
- One codebase, 14 weeks, nothing due now
- Tokens: text → IDs → embedding rows
- Next-token loss: chain rule, then cross-entropy

▼ What it takes to be productive from day 1

Being productive from day 1 takes four things.

You practice in the loop that this course runs every week. You start from a specification, build an implementation, test and measure it, and use the feedback from the tutorial to decide what to change next. The loop closes the gap between knowing about a system and being able to diagnose one. All of the work extends one config-driven codebase over 14 weeks. Week 1 sets up the concepts and the environment, and nothing is due this week.

You know where the field stands. Its core architecture, the Transformer, dates from 2017. Since then the field has moved fast, through seven eras from the Transformer to today's agents, and this course builds a piece of most of them.

You derisk every decision before you scale it. Reasoning alone cannot tell you whether a change helps, so you declare the experiment before the run. You state a hypothesis and test one intended change against a trusted baseline, with a matched budget of tokens and compute. You fix the evaluation set and a validated metric, plan a noise check, and declare in advance the threshold for scaling up, revising or stopping. When every run is expensive, this is how a few runs still give you a reliable answer.

And you understand the one objective that everything else builds on. A tokenizer cuts text into pieces and maps each piece to an integer ID, and each ID selects one row of the embedding table. How the tokenizer segments text changes the sequence length, and its vocabulary size changes the size of the embedding table and the output layer. Bonus 1, which is required reading, covers tokenization in depth. At every position the model produces logits, and softmax turns them into a distribution over the vocabulary. The chain rule factors the probability of a sequence into next-token probabilities, an exact identity, and training minimizes the cross-entropy loss, the mean negative log-probability of the realized next tokens.

That is the goal of this course: to be productive from day 1. The next 14 weeks build the skills it takes. Week 2 turns the next-token objective into one trainable Transformer block.

35

To-dos until next week

Muddiest point

Do it right now! (2 min)

Intro survey

Do it at home.

Once you have a spot in the class

- Pick a tutorial slot on ILIAS
- Sign up for the cluster: takes days
- How-to for the cluster follows tomorrow

Before the tutorial

- Read Bonus 1: tokenization
- Think about both sets of reflection questions
- Write down one question to discuss

Week 2 turns the next-token objective into one trainable Transformer block.

Reflection questions

1. A team can explain its training system on paper, but its first multi-GPU run is unstable. How would you use the course's specification–implementation–measurement–feedback loop to decide what to inspect or change next? What evidence would keep the process from becoming trial and error?
2. A hospital wants a model that drafts discharge letters in German, and patient records may not leave its servers. Before anyone trains a model, write the decision record you would give the hospital: how you would decide what quality is good enough, how you would find out what the open models the hospital can run on site achieve, how you would tell a real difference from noise on a test set of a few hundred letters, and which result would make you recommend training your own model, adapting an existing one, or stopping.
3. A data-mixture change improves loss in a small, short-context ablation. What must remain comparable before treating that result as evidence for a larger, longer-context run, and what finding would make you withhold the scale-up decision?
4. Two tokenizers represent the same corpus with different average sequence lengths and vocabulary sizes. Trace two downstream consequences and design an evaluation that could justify choosing one for a constrained deployment.
5. A model's pretraining loss improves smoothly, but a benchmark gain appears under one prompt and threshold and disappears under another. What can the team conclude about prediction and capability, and what remains uncertain?

References

37

References: Compute and experimental engineering

- Ben Allal, L., et al. (2025). The Smol Training Playbook. Hugging Face.
- Ferlazzo, L., Ericsson, K. A., & Pool, R. (2016). Peak: An Interview with Anders Ericsson & Robert Pool. Education Week.
- Grattafiori, A., Dubey, A., Jauhri, A., et al. (2024). The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783.
- Hadsell, R. (2026, October 7). Remarks on the Panel on AI Innovation: Do We Need Open Frontier AI?. Frontier AI Innovation Summit, Stuttgart.
- Khandelwal, A., Yun, T., Nayak, N. V., Merullo, J., Bach, S. H., Sun, C., & Pavlick, E. (2025). $100K or 100 Days: Trade-offs when Pre-Training with Academic Resources. COLM.
- Open Source Initiative. (2024). The Open Source AI Definition 1.0.

38

References: Objective and tokenization

- Bakouch, E., et al. (2025, July 8). SmolLM3: smol, multilingual, long-context reasoner. Hugging Face.
- Delétang, G., et al. (2024). Language Modeling Is Compression. ICLR.
- Ouyang, L., et al. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS.
- PleIAs. (2025). Monad tokenizer. Hugging Face.
- Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI.
- Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are Emergent Abilities of Large Language Models a Mirage?. NeurIPS.
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.

39

References: Transformer and scaling foundations

- Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.
- Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.

References: LLM history and open models

- Baek, C., et al. (2026). A.X K2 Technical Report. arXiv:2608.30181.
- Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. NeurIPS.
- Choi, E., et al. (2026). K-EXAONE 2.0 Technical Report. arXiv:2608.04505.
- DeepSeek-AI, et al. (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437.
- DeepSeek-AI, et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Nature.
- DeepSeek-AI. (2026). DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348.
- Ettinger, A., et al. (2025). Olmo 3. arXiv:2512.13961.
- Gemma Team. (2026). Gemma 4 Technical Report. arXiv:2607.02770.
- GLM Team. (2026). GLM-5: From Vibe Coding to Agentic Engineering. arXiv:2602.15763.
- Greenblatt, R., Cotra, A., & Wijk, H. (2026, August 26). Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. METR.
- Groeneveld, D., et al. (2024). OLMo: Accelerating the Science of Language Models. ACL.
- Hoffmann, J., et al. (2022). An empirical analysis of compute-optimal large language model training. Advances in Neural Information Processing Systems 35 (NeurIPS 2022).
- Hu, K. (2023, February 1). ChatGPT sets record for fastest-growing user base - analyst note. Reuters.
- Huang, A., et al. (2026). Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters. arXiv:2602.10604.
- Hugging Face. (2026, July 16). Security incident disclosure, July 2026.
- Kimi Team. (2026). Kimi K3: Open Frontier Intelligence. arXiv:2607.24653.
- Kwa, T., et al. (2025). Measuring AI Ability to Complete Long Tasks. NeurIPS.
- Lai, X., et al. (2026). MiniMax Sparse Attention. arXiv:2606.13392.
- Lim, J., et al. (2026). Motif 3: Technical Report. arXiv:2608.09119.
- Liu, A. H., et al. (2026). Ministral 3. arXiv:2601.08584.
- Meinhardt, C., Nong, S., Webster, G., Hashimoto, T., & Manning, C. D. (2025). Beyond DeepSeek: China's Diverse Open-Weight AI Ecosystem and Its Policy Implications. Stanford HAI and DigiChina issue brief.
- Meituan LongCat Team. (2025). LongCat-Flash Technical Report. arXiv:2509.01322.
- Meta Superintelligence Labs. (2026, August 10). Introducing Muse Glimmer: An Open Agentic Model That Runs on Your Device. Meta AI Research.
- METR. (2026). Task-Completion Time Horizons of Frontier AI Models. Time Horizon 1.1, updated May 8, 2026.
- NVIDIA. (2025). Nemotron 3 Nano: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. arXiv:2512.20848.
- OpenAI. (2022, November 30). Introducing ChatGPT.
- OpenAI. (2024, September 12). Learning to reason with LLMs.
- OpenAI. (2025). gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925.
- OpenAI. (2026, July 21). OpenAI and Hugging Face partner to address security incident during model evaluation.
- Poolside Team, Abadji, J., et al. (2026). Laguna M.1/XS.2 Technical Report. arXiv:2605.27605.
- Qwen Team. (2026). Qwen3.5-397B-A17B. Hugging Face model card.
- Radford, A., et al. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI.
- Singh, V., et al. (2026). Arcee Trinity Large Technical Report. arXiv:2602.17004.

- Thinking Machines Lab. (2026, July 15). Inkling: Our Open-Weights Model.
- Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.
- Wei, J., et al. (2022). Emergent Abilities of Large Language Models. TMLR.
- Xiaomi LLM-Core Team. (2026). MiMo-V2-Flash Technical Report. arXiv:2601.02780.
- Yang, A., et al. (2025). Qwen3 Technical Report. arXiv:2505.09388.

Image credits

- Hevelius, J. (1673). Johannes and Elisabetha Hevelius observing with a large sextant. *Machinae coelestis*, Fig. M. Library of Congress. Public domain.
- Collaert, J., after Stradanus (1600). Title plate of the new inventions of modern times. *Nova Reperta*, Title plate. The Metropolitan Museum of Art. CC0.
- Agricola, G. (1556). Water-driven stamp mill crushing ore. *De re metallica*. National Library of Poland. Public domain.
- Kepler, J. (1596). Model of the planetary spheres. *Mysterium Cosmographicum*, Plate III. Royal Danish Library. Public domain.
- Hevelius, J. (1673). Pivoting astronomical quadrant beside two clocks. *Machinae coelestis*, Fig. D. Google Books, via Wikimedia Commons. Public domain.
- Anonymous (1746). Vaucanson's automata: flute player, drummer and duck. *Jouets mécaniques de Vaucanson*. Bibliothèque nationale de France. Public domain.
- Agricola, G. (1556). Mine shafts and hoisting gear. *De re metallica*. National Library of Poland. Public domain.

Answers

A1 · Derisking cost more than half the main run

Ablations and debugging	Main run
161,280 GPU-hours	276,480 GPU-hours

- 58% of the main run, 37% of both (Ben Allal et al., 2025)
- Small runs must test the same decision

▼ **Why derisking is more than running a smaller model**

To derisk a decision is to test it at small scale before a full run spends its compute on it. A small run helps only when it tests the same decision as the large one.

The playbook gives the compute breakdown of SmolLM3's pretraining. Ablations and debugging took 161,280 GPU-hours, including the compute spent on an unexpected scaling issue that forced a restart of training, and the main pretraining run took 276,480 GPU-hours (Ben Allal et al., 2025). The two percentages answer different questions, because they divide by different amounts:

```
161280 / 276480 = 0.5833… = 58.3% of main-run compute
161280 / (161280 + 276480) = 0.368421… = 36.8% of the two amounts combined
```

The first divides by the main run alone: ablations and debugging cost 58% as much as the main run. The second divides by the sum of both amounts: 37% of the two together, `36.84%` rounded to a whole number, went to ablations and debugging.

When compute is scarce, spending more than a third of it on small runs can look wasteful. The SmolLM3 team spent that share anyway. We read it as the price of a main run whose decisions small runs had already checked.

A small result carries over to a larger run only if the larger run keeps what made the comparison informative: the optimizer, the peak learning rate, the global-batch schedule and the early-step behavior. Changing the context length changes the sequence length and the memory that attention needs. Changing the model scale can change how optimization behaves. A shift of domain changes the distribution the model learns. Estimate the metric noise instead of assuming it is negligible. Listing these limits does not settle whether a particular result transfers.

Package every result, a failure included, as a record you can reuse: the configuration, the code and data revisions, the seed plan, the logs, the metric outputs and the decision you took. A record of a negative result or a failed run can keep a later full-scale run from repeating it, and from relying on hindsight to judge what went wrong.

Back to the question

A2 · Uncertainty depends on the context

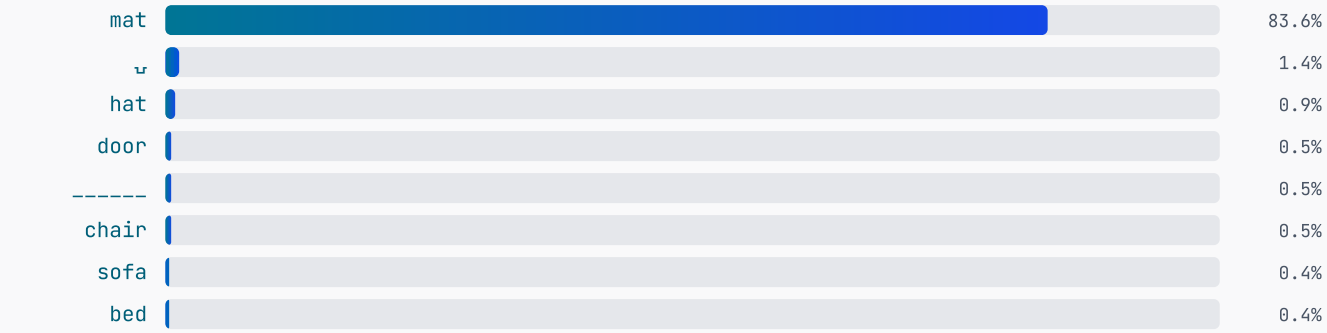
Next-token probabilities measured with SmolLM3-3B-Base, an open 3-billion-parameter base model, in October 2026. Its tokenizer is not the course tokenizer, so the candidate pieces differ from the ones you will see in this course. ␣ marks a token that is a non-breaking space.

The cat sat on the | My favorite color is | import torch import torch… | 2+2=

Prompt:

The cat sat on the

P(next token | prompt):



Remaining probability mass is spread across unlisted token candidates.

- Most uncertain: the favorite color
- Fixed phrase: one confident answer

Next-token probabilities measured with SmolLM3-3B-Base, an open 3-billion-parameter base model, in October 2026. Its tokenizer is not the course tokenizer, so the candidate pieces differ from the ones you will see in this course. ␣ marks a token that is a non-breaking space.

The cat sat on the | My favorite color is | import torch import torch… | 2+2=

Prompt:

My favorite color is|

P(next token | prompt):



Remaining probability mass is spread across unlisted token candidates.

The color prompt: its top candidate, “ blue”, gets only about 16%, and the eight listed candidates together hold about half of the probability.

▼ Reading a measured next-token distribution

The widget shows a base model's next-token distribution for one context at a time, `P(next token | prompt)`. This is the distribution that next-token training scores at that context. Each prompt button sets the context and with it the candidates, their percentages and the bars. The code prompt has the line `import torch` before `import torch.nn as`. For each prompt the widget lists the most probable candidates, eight for the two sentence prompts and five for the code and arithmetic prompts. One of them, " _____" for the cat prompt, is a run of underscores, probably the blank that quiz-like documents leave to fill in. The printed percentages are absolute probabilities, and so are the bars: a bar across the full width would mean 100%. A confident prediction therefore shows one long bar, and a spread one shows only short bars, so you can compare prompts by the bar lengths alone.

The values were measured with SmolLM3-3B-Base (Bakouch et al., 2025). The numbers hold for this model and its tokenizer.

Judge the uncertainty here by how much probability the top candidate leaves to the others. Two of the prompts are open English sentences, and they behave in opposite ways. The cat prompt is confident: " mat" gets about 84%, and no other listed candidate reaches 2%. "The cat sat on the mat" is a fixed phrase, and we read the 84% as the model having seen it many times in its training text. The color prompt is the most uncertain of the four. Its top candidate, " blue", gets only about 16%, followed by "

green" (9.1%), " red" (8.0%), " purple" (4.6%) and " yellow" (4.2%). Each of these is a plausible way to finish the sentence, and the eight listed candidates together hold only about half of the probability. The rest lies on tokens outside the list.

The other two prompts each have one clear winner. The code prompt gives " nn" more than 99%. After `2+2=`, "4" gets about 65%, but "5" still gets about 16%, and "2" and "3" get 3.6% and 3.2%. We read the 16% on "5" as the model having seen "2+2=5" in jokes and quotations: a base model continues a document the way its training text does, and nothing in that objective checks the arithmetic. Post-training can teach a model to answer a question instead of continuing a document (Weeks 12 and 13).

The same objective, with the same model, gives a confident distribution for one context and a spread one for another. What sets the uncertainty is the context, not the kind of text: an open sentence is not always uncertain.

Back to the question